

Fichas de entrenamiento

Categoría Gigabyte

7.º · 8.º · 9.º grado · nivel avanzado, con programación

20 desafíos de pensamiento computacional · Fase inicial

CÓMO SE RINDE

- La fase inicial son **9 desafíos en 60 minutos**, en duplas.
- Cada desafío tiene **una sola respuesta correcta** entre cuatro opciones.
- No hace falta saber programar ni usar la computadora: se resuelven razonando.
- Si un problema se traba, conviene pasar al siguiente y volver después.
- Las respuestas correctas y las explicaciones están al final del cuadernillo.

DESAFÍO 1 FÁCIL DEPURACIÓN; DIFERENCIA ENTRE ASIGNAR Y ACUMULAR

Depuración: la suma que no suma

Esta función debería devolver la suma de todos los números pares de un arreglo, pero tiene un error en una sola línea:

```
funcion sumaPares(A, n)
  variables s, i
  s <- 0
  para i desde 1 hasta n hacer
    si A[i] mod 2 = 0 entonces
      s <- A[i]
    fin si
  fin para
  devolver s
fin funcion

mostrar sumaPares([4, 7, 2, 9, 6], 5)
```

¿Qué devuelve el programa tal como está escrito, y cuál es la corrección?

- ☐ A Devuelve 12; hay que cambiar «mod 2 = 0» por «mod 2 <> 0»
- ☐ B Devuelve 4; hay que inicializar $s \leftarrow A[1]$
- ☐ C Devuelve 22; hay que cambiar «hasta n» por «hasta $n-1$ »
- ☐ D Devuelve 6; hay que cambiar $s \leftarrow A[i]$ por $s \leftarrow s + A[i]$

DESAFÍO 2**FÁCIL****VARIABLES Y ACUMULADORES; CONDICIONALES DENTRO DE UN BUCLE**

El acumulador caprichoso

Ana escribió el siguiente programa. El operador `mod` devuelve el resto de la división entera.

```
variables suma, i
suma <- 0
para i desde 1 hasta 10 hacer
  si i mod 3 = 0 entonces
    suma <- suma + i
  sino
    suma <- suma - 1
  fin si
fin para
mostrar suma
```

¿Qué valor muestra el programa?

- ☐ A 18
- ☐ B 8
- ☐ C 25
- ☐ D 11

DESAFÍO 3**FÁCIL****CONDICIONALES ANIDADOS Y ORDEN DE EVALUACIÓN**

La máquina de colores

Una máquina pinta cada número de un color según esta función:

```
funcion clasificar(n)
  si n mod 2 = 0 y n mod 3 = 0 entonces
    devolver "AZUL"
  sino
    si n mod 2 = 0 entonces
      devolver "VERDE"
    sino
      si n mod 3 = 0 entonces
        devolver "ROJO"
      sino
        devolver "GRIS"
    fin si
  fin si
fin funcion
```

¿Qué colores devuelve la máquina para los números 6, 8, 9 y 11, en ese orden?

- ☐ A AZUL, VERDE, ROJO, GRIS
- ☐ B VERDE, VERDE, ROJO, GRIS
- ☐ C AZUL, VERDE, GRIS, ROJO
- ☐ D ROJO, AZUL, VERDE, GRIS

Buscando en la biblioteca

El arreglo A está ordenado y sus posiciones se numeran del 1 al 10:

```
A = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
```

```
funcion buscar(A, x)
  variables izq, der, medio
  izq <- 1
  der <- 10
  mientras izq <= der hacer
    medio <- (izq + der) div 2
    si A[medio] = x entonces
      devolver medio
    fin si
    si A[medio] < x entonces
      izq <- medio + 1
    sino
      der <- medio - 1
    fin si
  fin mientras
  devolver 0
fin funcion

mostrar buscar(A, 23)
```

¿Qué valores de A se comparan con 23, en orden, y qué devuelve la función?

- ☐ A 16, 56, 23 y devuelve 6
 - ☐ B 16, 38, 23 y devuelve 6
 - ☐ C 16, 56, 38, 23 y devuelve 23
 - ☐ D 2, 5, 8, 12, 16, 23 y devuelve 6
-

DESAFÍO 5 **MEDIA** **DEPURACIÓN; INICIALIZACIÓN DE UNA VARIABLE ACUMULADORA****Depuración: el máximo que falla**

Un estudiante escribió esta función para encontrar el elemento más grande de un arreglo de n elementos (posiciones 1 a n):

```
funcion maximo(A, n)
  variables m, i
  m <- 0
  para i desde 1 hasta n hacer
    si A[i] > m entonces
      m <- A[i]
    fin si
  fin para
  devolver m
fin funcion
```

La función funciona bien casi siempre, pero tiene un error.

¿Con cuál de estos arreglos devuelve un resultado INCORRECTO?

- ☐ A [3, -7, 12, 0]
- ☐ B [-5, -2, -9, -1]
- ☐ C [1, 2, 3, 4]
- ☐ D [10, 10, 10]

DESAFÍO 6 **MEDIA** **ALGORITMO DE EUCLIDES; TRAZA DE BUCLE CON DOS VARIABLES****El algoritmo de Euclides**

Este es el algoritmo de Euclides para el máximo común divisor:

```
funcion mcd(a, b)
  variables r
  mientras b <> 0 hacer
    r <- a mod b
    a <- b
    b <- r
  fin mientras
  devolver a
fin funcion

mostrar mcd(84, 30)
```

¿Cuántas veces se ejecuta el cuerpo del «mientras» y qué valor se devuelve?

- ☐ A 2 veces, devuelve 6
- ☐ B 3 veces, devuelve 6
- ☐ C 3 veces, devuelve 2
- ☐ D 4 veces, devuelve 12

El dibujo escalonado

Este programa imprime cuatro líneas de texto. El símbolo `+` concatena (pega) textos.

```
para i desde 1 hasta 4 hacer
  linea <- ""
  para j desde 1 hasta i hacer
    si (i + j) mod 2 = 0 entonces
      linea <- linea + "#"
    sino
      linea <- linea + "."
  fin si
  fin para
  mostrar linea
fin para
```

¿Qué dibujo se obtiene?

- ☐ A `# / .# / #.# / .#.#`
- ☐ B `# / #. / #.# / #.#.`
- ☐ C `./ .# / .#. / .#.#`
- ☐ D `# / ## / ### / ####`

El vuelo de Collatz

La secuencia de Collatz aplica esta regla hasta llegar a 1:

```
variables n, pasos
n <- 11
pasos <- 0
mientras n <> 1 hacer
  si n mod 2 = 0 entonces
    n <- n div 2
  sino
    n <- 3 * n + 1
  fin si
  pasos <- pasos + 1
fin mientras
mostrar pasos
```

¿Cuántos pasos muestra el programa cuando se empieza en 11?

- ☐ A 11
- ☐ B 12
- ☐ C 14
- ☐ D 52

La función misteriosa

El operador `div` es la división entera (descarta el resto) y `mod` es el resto.

```
funcion misterio(n)
  si n < 10 entonces
    devolver n
  sino
    devolver (n mod 10) + misterio(n div 10)
  fin si
fin funcion

mostrar misterio(4729)
```

¿Qué muestra el programa?

- ☐ A 9
- ☐ B 22
- ☐ C 4729
- ☐ D 4

La pila y la cola

Una máquina clasificadora recibe los números del 1 al 6 en orden. Los pares van a una PILA (el último que entra es el primero que sale) y los impares a una COLA (el primero que entra es el primero que sale). Después se vacía primero la cola y luego la pila.

```
pila <- vacía
cola <- vacía
para n desde 1 hasta 6 hacer
  si n mod 2 = 0 entonces
    apilar(pila, n)
  sino
    encolar(cola, n)
  fin si
fin para

mientras cola no esté vacía hacer
  mostrar desencolar(cola)
fin mientras
mientras pila no esté vacía hacer
  mostrar desapilar(pila)
fin mientras
```

¿Qué secuencia muestra la máquina?

- ☐ A 1, 3, 5, 2, 4, 6
- ☐ B 5, 3, 1, 6, 4, 2
- ☐ C 2, 4, 6, 1, 3, 5
- ☐ D 1, 3, 5, 6, 4, 2

Los pares de castores

En una represa hay 5 castores numerados del 1 al 5. Este programa cuenta cuántos saludos ocurren.

```
variables cuenta, i, j
cuenta <- 0
para i desde 1 hasta 5 hacer
  para j desde i hasta 5 hacer
    cuenta <- cuenta + 1
  fin para
fin para
mostrar cuenta
```

Observá con cuidado el valor inicial del bucle interno.

¿Qué muestra el programa?

- ☐ A 10
- ☐ B 15
- ☐ C 20
- ☐ D 25

Máximo, mínimo y cuántos cambios

Este programa busca el mayor y el menor de un arreglo (posiciones 1 a 7):

```
A = [7, 2, 9, 4, 9, 1, 6]
max <- A[1]
min <- A[1]
para i desde 2 hasta 7 hacer
  si A[i] > max entonces
    max <- A[i]
  fin si
  si A[i] < min entonces
    min <- A[i]
  fin si
fin para
mostrar max - min
```

¿Cuánto vale $\text{max} - \text{min}$ y cuántas veces en total se ejecutan las líneas de asignación que están dentro de los «si»?

- ☐ A 8 y 3 asignaciones
- ☐ B 8 y 4 asignaciones
- ☐ C 9 y 3 asignaciones
- ☐ D 8 y 2 asignaciones

Una sola pasada de burbuja

Este fragmento realiza UNA pasada del ordenamiento de burbuja sobre un arreglo de 5 elementos (posiciones 1 a 5):

```
A = [5, 1, 4, 2, 8]

para i desde 1 hasta 4 hacer
  si A[i] > A[i+1] entonces
    t <- A[i]
    A[i] <- A[i+1]
    A[i+1] <- t
  fin si
fin para
mostrar A
```

¿Cómo queda el arreglo al terminar?

- ☐ A [1, 2, 4, 5, 8]
- ☐ B [1, 4, 2, 5, 8]
- ☐ C [1, 5, 4, 2, 8]
- ☐ D [5, 1, 4, 2, 8]

Contando operaciones

La variable **c** cuenta cuántas veces se ejecuta el cuerpo del bucle interno:

```
variables c, i, j, n
n <- 16
c <- 0
para i desde 1 hasta n hacer
  j <- 1
  mientras j <= n hacer
    c <- c + 1
    j <- j * 2
  fin mientras
fin para
mostrar c
```

¿Qué valor muestra?

- ☐ A 32
- ☐ B 256
- ☐ C 64
- ☐ D 80

Depuración: el arreglo que no se invierte

Este código intenta invertir un arreglo de n elementos (posiciones 1 a n), pero algo anda mal:

```
A = [1, 2, 3, 4, 5]
n <- 5
para i desde 1 hasta n hacer
  t <- A[i]
  A[i] <- A[n + 1 - i]
  A[n + 1 - i] <- t
fin para
mostrar A
```

¿Qué muestra el programa y por qué?

- ☐ A [5, 4, 3, 2, 1], el código es correcto
- ☐ B [5, 2, 3, 4, 1], solo intercambia los extremos
- ☐ C [1, 2, 3, 4, 5], porque cada par se intercambia dos veces
- ☐ D [1, 4, 3, 2, 5], porque el elemento del medio queda mal

El juego de la ronda

Seis chicos están formados en una cola, numerados del 1 al 6 (el 1 está adelante). Se repite el siguiente juego hasta que quede uno solo:

```
cola = [1, 2, 3, 4, 5, 6]

mientras longitud(cola) > 1 hacer
  x <- desencolar(cola)
  encolar(cola, x)
  y <- desencolar(cola)
fin mientras
mostrar frente(cola)
```

Es decir: el primero de la fila se va al final (se salva por ahora) y el siguiente queda eliminado.

¿Qué número muestra el programa?

- ☐ A 3
- ☐ B 4
- ☐ C 5
- ☐ D 6

Ordenamiento por inserción: contando movimientos

Este es el ordenamiento por inserción sobre un arreglo de 5 elementos (posiciones 1 a 5):

```
A = [4, 3, 5, 1, 2]
movimientos <- 0
para i desde 2 hasta 5 hacer
  actual <- A[i]
  j <- i - 1
  mientras j >= 1 y A[j] > actual hacer
    A[j+1] <- A[j]
    movimientos <- movimientos + 1
    j <- j - 1
  fin mientras
  A[j+1] <- actual
fin para
mostrar movimientos
```

¿Cuántos movimientos (líneas $A[j+1] \leftarrow A[j]$ ejecutadas) cuenta el programa?

- ☐ A 4
- ☐ B 5
- ☐ C 7
- ☐ D 10

Potencia rápida

Esta función recursiva calcula potencias de manera astuta:

```
funcion potencia(b, e)
  si e = 0 entonces
    devolver 1
  fin si
  si e mod 2 = 0 entonces
    m <- potencia(b, e div 2)
    devolver m * m
  sino
    devolver b * potencia(b, e - 1)
  fin si
fin funcion

mostrar potencia(2, 10)
```

¿Qué valor se muestra y cuántas veces en total se entra a la función potencia (contando la primera llamada)?

- ☐ A 1024 y 6 llamadas
- ☐ B 1024 y 10 llamadas
- ☐ C 100 y 6 llamadas
- ☐ D 1024 y 11 llamadas

DESAFÍO 19 **DIFÍCIL** **ÁRBOLES BINARIOS Y RECORRIDOS RECURSIVOS****Recorriendo el árbol**

Un árbol binario se guarda con tres arreglos. La raíz es el nodo 1 y el 0 significa «no hay hijo».

nodo:	1	2	3	4	5	6	7	8
valor:	8	3	10	1	6	14	4	7
izq:	2	4	0	0	7	0	0	0
der:	3	5	6	0	8	0	0	0

Se aplica este recorrido:

```
funcion recorrer(i)
  si i = 0 entonces
    devolver
  fin si
  recorrer(izq[i])
  recorrer(der[i])
  mostrar valor[i]
fin funcion

recorrer(1)
```

¿Qué secuencia de valores se muestra?

- ☐ A 8, 3, 1, 6, 4, 7, 10, 14
- ☐ B 1, 3, 4, 6, 7, 8, 10, 14
- ☐ C 1, 4, 7, 6, 3, 14, 10, 8
- ☐ D 8, 10, 14, 3, 6, 7, 4, 1

DESAFÍO 20 **DIFÍCIL** **RECURSIÓN, ÁRBOL DE LLAMADAS Y COSTO DE UN ALGORITMO****¿Cuánto trabaja Fibonacci?**

Esta es la definición recursiva clásica de Fibonacci:

```
funcion fib(n)
  si n <= 1 entonces
    devolver n
  fin si
  devolver fib(n-1) + fib(n-2)
fin funcion
```

Contamos UNA llamada cada vez que se entra a la función `fib`, incluida la primera.

¿Cuántas llamadas en total se realizan al calcular fib(6)?

- ☐ A 6
- ☐ B 13
- ☐ C 15
- ☐ D 25

Hoja de respuestas · Gigabyte

Marcá con una X la opción elegida en cada desafío.

Nombres de la dupla · Grado · Colegio · Fecha

N.º	RESPUESTA	N.º	RESPUESTA
1	A B C D	11	A B C D
2	A B C D	12	A B C D
3	A B C D	13	A B C D
4	A B C D	14	A B C D
5	A B C D	15	A B C D
6	A B C D	16	A B C D
7	A B C D	17	A B C D
8	A B C D	18	A B C D
9	A B C D	19	A B C D
10	A B C D	20	A B C D

Soluciones · Gigabyte

Hoja del docente. No repartir junto con las fichas.

N.º	DESAFÍO	RESP.	NIVEL	CONCEPTO
1	Depuración: la suma que no suma	D	fácil	Depuración; diferencia entre asignar y acumular
2	El acumulador caprichoso	D	fácil	Variables y acumuladores; condicionales dentro de un bucle
3	La máquina de colores	A	fácil	Condicionales anidados y orden de evaluación
4	Buscando en la biblioteca	A	media	Búsqueda binaria; división entera
5	Depuración: el máximo que falla	B	media	Depuración; inicialización de una variable acumuladora
6	El algoritmo de Euclides	B	media	Algoritmo de Euclides; traza de bucle con dos variables
7	El dibujo escalonado	A	media	Bucles anidados, construcción de cadenas y paridad
8	El vuelo de Collatz	C	media	Bucle «mientras» con condición no obvia; algoritmo de Collatz
9	La función misteriosa	B	media	Recursión con caso base; división entera y módulo
10	La pila y la cola	D	media	Estructuras de datos: pilas (LIFO) y colas (FIFO)
11	Los pares de castores	B	media	Bucles anidados dependientes
12	Máximo, mínimo y cuántos cambios	A	media	Máximo y mínimo; comparaciones estrictas; traza de bucle
13	Una sola pasada de burbuja	B	media	Algoritmos de ordenamiento: burbuja, una pasada
14	Contando operaciones	D	difícil	Complejidad básica: bucle logarítmico dentro de uno lineal
15	Depuración: el arreglo que no se invierte	C	difícil	Depuración; arreglos, intercambio y rango de un bucle
16	El juego de la ronda	C	difícil	Simulación con colas (FIFO); problema de Josefo
17	Ordenamiento por inserción: contando movimientos	C	difícil	Ordenamiento por inserción; contadores en bucles anidados
18	Potencia rápida	A	difícil	Recursión y eficiencia; exponenciación rápida

19	Recorriendo el árbol	C	difícil	Árboles binarios y recorridos recursivos
20	¿Cuánto trabaja Fibonacci?	D	difícil	Recursión, árbol de llamadas y costo de un algoritmo

Explicaciones

1. Depuración: la suma que no suma — respuesta D) Devuelve 6; hay que cambiar $s \leftarrow A[i]$ por $s \leftarrow s + A[i]$

Como la línea dice $s \leftarrow A[i]$ (asignación) en lugar de $s \leftarrow s + A[i]$ (acumulación), cada par PISA el valor anterior en vez de sumarse. Trazando [4, 7, 2, 9, 6]: el 4 es par, $s = 4$; el 7 no; el 2 es par, $s = 2$; el 9 no; el 6 es par, $s = 6$. Devuelve 6, que es simplemente el ÚLTIMO par del arreglo. El resultado correcto es $4+2+6 = 12$, y se obtiene acumulando.

Fuente: Variante original estilo Bebras (verificado por ejecución)

2. El acumulador caprichoso — respuesta D) 11

Entre 1 y 10 hay exactamente 3 múltiplos de 3: el 3, el 6 y el 9. Para ellos se ejecuta la rama «entonces» y se suman: $3+6+9 = 18$. Los otros 7 números (1, 2, 4, 5, 7, 8, 10) caen en la rama «sino» y cada uno resta 1: -7 . Total: $18 - 7 = 11$. El error típico es olvidar la rama «sino» y responder 18.

Fuente: Variante original estilo Bebras (verificado por ejecución)

3. La máquina de colores — respuesta A) AZUL, VERDE, ROJO, GRIS

$n=6$: es par y múltiplo de 3, entra en la primera condición → AZUL. $n=8$: es par pero no múltiplo de 3 → VERDE. $n=9$: no es par, salta a la tercera condición y es múltiplo de 3 → ROJO. $n=11$: no es par ni múltiplo de 3 → GRIS. La clave es que el orden de los «sí» importa: el caso AZUL debe probarse primero, porque si no, el 6 se clasificaría como VERDE.

Fuente: Variante original estilo Bebras (verificado por ejecución)

4. Buscando en la biblioteca — respuesta A) 16, 56, 23 y devuelve 6

Vuelta 1: $izq=1$, $der=10$, $medio = 11 \div 2 = 5$, $A[5] = 16$. Como $16 < 23 \rightarrow izq = 6$. Vuelta 2: $izq=6$, $der=10$, $medio = 8$, $A[8] = 56$. Como $56 > 23 \rightarrow der = 7$. Vuelta 3: $izq=6$, $der=7$, $medio = 6$, $A[6] = 23$: ¡encontrado! Devuelve la POSICIÓN 6, no el valor. Se compararon 16, 56 y 23: solo 3 comparaciones para 10 elementos. La opción D describe una búsqueda lineal.

Fuente: Variante original estilo Bebras (verificado por ejecución)

5. Depuración: el máximo que falla — respuesta B) [-5, -2, -9, -1]

El error está en inicializar m con 0 en lugar de con $A[1]$. Con el arreglo $[-5, -2, -9, -1]$ ningún elemento es mayor que 0, así que la condición nunca se cumple, m nunca cambia y la función devuelve 0, aunque el 0 ni siquiera está en el arreglo. La respuesta correcta debería ser -1 . En las otras opciones hay al menos un valor mayor que 0, así que el resultado sale bien por casualidad. La corrección es escribir $m \leftarrow A[1]$ y recorrer desde $i = 2$.

Fuente: Variante original estilo Bebras (verificado por ejecución)

6. El algoritmo de Euclides — respuesta B) 3 veces, devuelve 6

Trazamos los pares (a, b): empieza (84, 30). Vuelta 1: $r = 84 \bmod 30 = 24 \rightarrow (30, 24)$. Vuelta 2: $r = 30 \bmod 24 = 6 \rightarrow (24, 6)$. Vuelta 3: $r = 24 \bmod 6 = 0 \rightarrow (6, 0)$. Ahora $b = 0$ y el bucle termina, devolviendo $a = 6$. Fueron 3 vueltas. Comprobación: $84 = 6 \times 14$ y $30 = 6 \times 5$, y 14 y 5 no tienen divisores comunes.

Fuente: Variante original estilo Bebras (verificado por ejecución)

7. El dibujo escalonado — respuesta A) # / .# / #.# / .#.#

La línea i tiene exactamente i caracteres, y en la posición j se pone «#» si $i+j$ es par y «.» si es impar. $i=1$: $1+1=2$ par $\rightarrow$ «#». $i=2$: 3 impar $\rightarrow$ «.», 4 par $\rightarrow$ «#» $\rightarrow$ «.#». $i=3$: 4 par $\rightarrow$ «#», 5 impar $\rightarrow$ «.», 6 par $\rightarrow$ «#» $\rightarrow$ «#.#». $i=4$: «.#.#». Las líneas impares empiezan con «#» y las pares con «.».

Fuente: Variante original estilo Bebras (verificado por ejecución)

8. El vuelo de Collatz — respuesta C) 14

La secuencia completa es: $11 \rightarrow 34 \rightarrow 17 \rightarrow 52 \rightarrow 26 \rightarrow 13 \rightarrow 40 \rightarrow 20 \rightarrow 10 \rightarrow 5 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$. Después del 11 aparecen 14 números más, y cada uno corresponde a una vuelta del bucle, o sea 14 pasos. Notá que 52 es el valor MÁS GRANDE que alcanza la secuencia, no la cantidad de pasos: ese es el distractor clásico.

Fuente: Variante original estilo Bebras (verificado por ejecución)

9. La función misteriosa — respuesta B) 22

La función va cortando el último dígito y sumándolo al resultado de llamarse a sí misma con el número sin ese dígito: $\text{misterio}(4729) = 9 + \text{misterio}(472) = 9 + 2 + \text{misterio}(47) = 9 + 2 + 7 + \text{misterio}(4)$. Como $4 < 10$, $\text{misterio}(4)$ devuelve 4. Sumando: $9 + 2 + 7 + 4 = 22$. La función calcula la suma de los dígitos.

Fuente: Variante original estilo Bebras (verificado por ejecución)

10. La pila y la cola — respuesta D) 1, 3, 5, 6, 4, 2

Los impares 1, 3, 5 entran a la cola en ese orden. Los pares 2, 4, 6 se apilan: 2 abajo, luego 4, y 6 arriba de todo. Al vaciar la cola sale en el MISMO orden en que entró: 1, 3, 5. Al vaciar la pila sale en orden INVERSO, empezando por el de arriba: 6, 4, 2. La secuencia final es 1, 3, 5, 6, 4, 2.

Fuente: Variante original estilo Bebras (verificado por ejecución)

11. Los pares de castores — respuesta B) 15

El bucle interno NO empieza siempre en 1: empieza en i . Entonces, para $i=1$ el bucle interno corre con $j=1..5$ (5 veces), para $i=2$ corre $j=2..5$ (4 veces), para $i=3$ son 3 veces, para $i=4$ son 2 veces y para $i=5$ es 1 vez. Total: $5+4+3+2+1 = 15$. La trampa es responder 25 (como si fueran dos bucles completos) o 10 (como si j empezara en $i+1$).

Fuente: Variante original estilo Bebras (verificado por ejecución)

12. Máximo, mínimo y cuántos cambios — respuesta A) 8 y 3 asignaciones

Arrancamos con $\max = \min = 7$. $i=2$ ($A=2$): $2 < 7 \rightarrow \min = 2$ (asignación 1). $i=3$ ($A=9$): $9 > 7 \rightarrow \max = 9$ (asignación 2). $i=4$ ($A=4$): ninguna. $i=5$ ($A=9$): la comparación es $9 > 9$, que es FALSA porque el operador es estrictamente mayor, así que NO hay asignación (acá está la trampa). $i=6$ ($A=1$): $1 < 2 \rightarrow \min = 1$ (asignación 3). $i=7$ ($A=6$): ninguna. Quedan $\max = 9$ y $\min = 1$, así que $\max - \min = 8$, con 3 asignaciones en total.

Fuente: Variante original estilo Bebras (verificado por ejecución)

13. Una sola pasada de burbuja — respuesta B) [1, 4, 2, 5, 8]

Vamos comparando vecinos e intercambiando si están desordenados. $i=1$: $5 > 1 \rightarrow [1, 5, 4, 2, 8]$. $i=2$: $5 > 4 \rightarrow [1, 4, 5, 2, 8]$. $i=3$: $5 > 2 \rightarrow [1, 4, 2, 5, 8]$. $i=4$: $5 > 8$ es falso, no hay intercambio. Resultado: $[1, 4, 2, 5, 8]$. Una sola pasada NO ordena el arreglo: solo garantiza que el elemento más grande quede al final.

Fuente: Variante original estilo Bebras (verificado por ejecución)

14. Contando operaciones — respuesta D) 80

El bucle interno no suma de a 1: DUPLICA j en cada vuelta. Con $n=16$, j toma los valores 1, 2, 4, 8 y 16 (todos ≤ 16) y luego 32, que ya no cumple la condición. Son 5 vueltas del bucle interno. El bucle externo se repite 16 veces y el interno se reinicia cada vez. Total: $16 \times 5 = 80$. La opción 256 = 16×16 sería la respuesta si el bucle interno avanzara de a 1: por eso duplicar en vez de sumar hace al algoritmo muchísimo más rápido cuando n crece.

Fuente: Variante original estilo Bebras (verificado por ejecución)

15. Depuración: el arreglo que no se invierte — respuesta C) [1, 2, 3, 4, 5], porque cada par se intercambia dos veces

El intercambio en sí está bien escrito, pero el bucle recorre TODO el arreglo en lugar de solo la primera mitad. Con $i=1$ se intercambian las posiciones 1 y 5 $\rightarrow [5, 2, 3, 4, 1]$; con $i=2$ las posiciones 2 y 4 $\rightarrow [5, 4, 3, 2, 1]$; con $i=3$ se intercambia la posición 3 consigo misma; con $i=4$ se vuelven a intercambiar 4 y 2, deshaciendo el trabajo $\rightarrow [5, 2, 3, 4, 1]$; con $i=5$ se vuelven a intercambiar 5 y 1 $\rightarrow [1, 2, 3, 4, 5]$. El arreglo queda igual que al principio. La corrección es que el bucle vaya desde 1 hasta $n \div 2$.

Fuente: Variante original estilo Bebras (verificado por ejecución)

16. El juego de la ronda — respuesta C) 5

Trazamos la cola vuelta por vuelta. $[1, 2, 3, 4, 5, 6]$: sale el 1 y vuelve al final $\rightarrow [2, 3, 4, 5, 6, 1]$; se elimina el 2 $\rightarrow [3, 4, 5, 6, 1]$. Sale el 3 y vuelve $\rightarrow [4, 5, 6, 1, 3]$; se elimina el 4 $\rightarrow [5, 6, 1, 3]$. Sale el 5 y vuelve $\rightarrow [6, 1, 3, 5]$; se elimina el 6 $\rightarrow [1, 3, 5]$. Sale el 1 y vuelve $\rightarrow [3, 5, 1]$; se elimina el 3 $\rightarrow [5, 1]$. Sale el 5 y vuelve $\rightarrow [1, 5]$; se elimina el 1 $\rightarrow [5]$. Queda el 5.

Fuente: Variante original estilo Bebras (verificado por ejecución)

17. Ordenamiento por inserción: contando movimientos — respuesta C) 7

$i=2$, $\text{actual}=3$: el 4 es mayor, se corre 1 lugar (1 movimiento) $\rightarrow [3, 4, 5, 1, 2]$. $i=3$, $\text{actual}=5$: el 4 no es mayor que 5 (0 movimientos). $i=4$, $\text{actual}=1$: se corren el 5, el 4 y el 3 (3 movimientos) $\rightarrow [1, 3, 4, 5, 2]$. $i=5$, $\text{actual}=2$: se corren el 5, el 4 y el 3, pero el 1 no es mayor que 2 y el bucle se detiene (3 movimientos) $\rightarrow [1, 2, 3, 4, 5]$. Total: $1+0+3+3 = 7$ movimientos.

Fuente: Variante original estilo Bebras (verificado por ejecución)

18. Potencia rápida — respuesta A) 1024 y 6 llamadas

Seguimos el exponente: potencia(2,10) → como 10 es par, llama a potencia(2,5) → como 5 es impar, llama a potencia(2,4) → potencia(2,2) → potencia(2,1) → potencia(2,0), que devuelve 1. Las llamadas son con exponentes 10, 5, 4, 2, 1 y 0: 6 en total. Los valores vuelven así: potencia(2,1)=2; potencia(2,2)=4; potencia(2,4)=16; potencia(2,5)=32; potencia(2,10)=32×32=1024. La versión ingenua necesitaría 11 llamadas: partir el exponente al medio ahorra trabajo.

Fuente: Variante original estilo Bebras (verificado por ejecución)

19. Recorriendo el árbol — respuesta C) 1, 4, 7, 6, 3, 14, 10, 8

Como el «mostrar» está DESPUÉS de las dos llamadas recursivas, se trata de un recorrido en POSTORDEN: primero el subárbol izquierdo, después el derecho y recién al final la raíz. El árbol es: raíz 8, con hijo izquierdo 3 y derecho 10; el 3 tiene hijos 1 y 6; el 6 tiene hijos 4 y 7; el 10 solo tiene hijo derecho 14. Postorden del subárbol del 3: 1, 4, 7, 6, 3. Del subárbol del 10: 14, 10. Finalmente la raíz 8. Secuencia: 1, 4, 7, 6, 3, 14, 10, 8. La opción A es el PREORDEN y la B es el INORDEN.

Fuente: Variante original estilo Bebras (verificado por ejecución)

20. ¿Cuánto trabaja Fibonacci? — respuesta D) 25

Si $L(n)$ es el total de llamadas para fib(n): $L(0)=1$ y $L(1)=1$, porque llegan al caso base sin llamar a nadie más. Para $n \geq 2$, $L(n) = 1 + L(n-1) + L(n-2)$. Entonces $L(2)=3$, $L(3)=5$, $L(4)=9$, $L(5)=15$ y $L(6)=25$. Ojo: 8 es el VALOR de fib(6), no la cantidad de llamadas; 15 es el total de llamadas para fib(5). Esto muestra por qué la versión recursiva sin memoria es tan lenta: recalcula lo mismo muchas veces.

Fuente: Variante original estilo Bebras (verificado por ejecución)

Material de entrenamiento para la Olimpiada Aguarandú-Bebras (OMAPA, Paraguay). Los desafíos identificados como «ficha oficial 2025» pertenecen a OMAPA; el resto son adaptaciones y variantes en estilo Bebras creadas para este cuadernillo. Uso educativo sin fines de lucro.